

1. 基础

1.1 须知

RE考虑以下函数返回值是不是写错了

c++11 有auto关键字, unordered_map、unordered_set

c++98 priority_queue

```
#define endl '\n'
int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
}

ios::sync_with_stdio(false)//提高cin读取速度，但不能用scanf了
while(cin>>a[n]>>b[n]) n++; //不确定会有几个方法读进来

cout << '\n'; //取代endl，评测卡endl
```

Fence 1

1.2 数据类型

数据类型	说明	字节	取值范围
int	整型	2	$(-32768 \sim 32767) - 2^{15} \sim 2^{15} - 1$
short int	短整型 (int可以省略)	2	$(-32768 \sim 32767) - 2^{15} \sim 2^{15} - 1$
long int	长整型 (int可以省略)	4	$(-2147483648 \sim 2147483647) - 2^{31} \sim 2^{31} - 1$
unsigned int	无符号整型	2	$(0 \sim 65535) 0 \sim 2^{16} - 1$
unsigned short int	无符号短整型 (int可以省略)	2	$(0 \sim 65535) 0 \sim 2^{16} - 1$
unsigned long int	无符号长整型 (int可以省略)	4	$(0 \sim 4294967295) 0 \sim 2^{32} - 1$

Image 1

数据类型	说明	字节	取值范围
float	单精度型	4	$-3.4 \times 10^{-38} \sim 3.4 \times 10^{38}$
double	双精度型	8	$-1.7 \times 10^{-308} \sim 1.7 \times 10^{308}$
long double	长双精度型	16	$-1.2 \times 10^{-4932} \sim 1.7 \times 10^{4932}$

Image 2

%d	带十进制整数
%c	单个字符
%s	字符串
%f	6位小数
%lf	对应double类型
%3.2f	3为小数点前保留，2为小数点后保留

Table 1

floor->x/2替代 ceil->(x+1)/2替代

1.3 注意事项

```
if (isdigit(s[i])) ans++; //判断是否是数字
if (islower(s[i])) ans++; //判断是否是小写字母
if (isupper(s[i])) ans++; //判断是否是大写字母
if (isalpha(s[i])) ans++; //判断是否为字母
if (isalnum(s[i]) != 0) ans++; //判断是否为大、小写英文字母和数字

int i, j;
scanf("%d%d", &i, &j);
for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]); // 逐个输入数组元素
}
printf("i = %d, j = %d\n", i, j);
```

Fence 2

设置无穷大 0x3f3f3f3f 1e9

常用库函数 memset(h, 0x3f, sizeof h); h为数组

unique去重（要先进行排序，保证重复元素相同）

把一个vector去重

int m=unique(a.begin(), a.end())-a.begin(); m返回去重后元素的数量

有序的容器：

1	1	2	3	3	4	4	4	5	6
---	---	---	---	---	---	---	---	---	---

unique处理过的容器：

unique	unique	unique	unique	unique	unique	迭代器指向的地址			
1	2	3	4	5	6	1	3	4	4

Image 3

sort（）函数

sort(begin, end, compare); begin第一个元素指针，end为最后一个元素下一个指针

默认从小到大排

```
int num[10] = {6,5,9,1,2,8,7,3,4,0};
sort(num, num+10);
for(int i=0; i<10; i++){
    cout<<num[i]<<" ";
}
return 0;
```

从大到小排-> greater()

自定义排序规则 写一个bool函数传入cmp

```
int a[MAX_SIZE];
bool cmp(int a, int b)
{
    return a > b;
}
sort(a + 1, a + n + 1, cmp);
```

1.4 初始化

```
memset(a, 0x3f, sizeof(a));
memset(head, 0, sizeof(int) * (n + 1));
vector<vector<int>> dp(100, vector<int>(100, -1)); // 100 x 100 的 dp 表, 全部初始化为 -1
```

1.5 常用数据结构

1.5.1 string

```
string s;
s.push_back('a');
s.pop_back();
//s.length(); s.empty(); s.clear(); string c = a + b;

string s = "abcdef"; //复杂度: O(len)
cout << s.substr(1, 3); // bcd pos, len. 不写len就截到末尾

string s = "hello world";
int pos = s.find("world"); //复杂度: O(n*m)
if (pos != string::npos) {
    cout << pos; // 6
}

string s = "abcd";
s.insert(2, "xx");
cout << s; // abxxcd

s.erase(pos, len); //从 pos 开始删除 len 个字符。 复杂度: O(n)
int l = 0;
while (l < s.size()) { //用指针记录替代erase
    l++;
}
```

```
sort(s.begin(), s.end()); //复杂度:  $O(n \log n)$ 
reverse(s.begin(), s.end()); //复杂度:  $O(n)$ 
```

Fence 4

1.5.2 队列queue

```
queue<int> q1;

//常用操作
q.push() //队尾插入
q.pop() //删除最后一个元素
q.size()
q.empty()
q.front() //返回第一个元素
q.back() //返回最后一个元素
```

Fence 5

1.5.3 栈stack

```
stack<int> q;

q.push(x) //队尾插入
q.top() //返回栈顶元素
q.pop()
q.size()
q.empty()
```

Fence 6

1.5.4 优先队列priority_queue

只操作头尾，大根堆头大尾小，小根堆反过来

```
priority_queue<int> q; //大根堆
priority_queue<int, vector<int>, greater<int>> q; //小根堆

q.top();
q.pop();
q.push();
```

Fence 7

1.5.5 vector

支持随机访问，但不支持 $O(1)$ 任意位置插入，增删一般在末尾进行

vector新用法

```
n=text1.length();
m=text2.length();
vector<vector<int>> f(n+1, vector<int>(m+1, 0)); //二维初始化

vector<vector<int>> ans;
vector<int> path;
```

Fence 8

```
vector<int> v;
v.resize(len); //重新分配len大小空间
vector<int> v(10);
vector<int> v(10,2);

struct rec{
    string a;
    int b;
};
vector<rec> c; //结构体也能存vector
v.push_back(x); //队尾加元素
v.pop_back(x); //删除最后一个元素
v.empty();
```

Fence 9

1.5.6 pair

pair排序规则：先first升序排，first相等看second

```
typedef pair<int,int> PII;
int main(){
    vector<PII> v(10);
    priority_queue<pair<int,int>> q;
    //循环输入输出
    for(int i=0;i<10;i++)
        v.pushback(t);
    for(int i=0;i<v.size();i++){
        cout<<v[i].first<<" "<<v[i].second<<endl;
    }
}
```

Fence 10

1.5.7 map

```
map<string,int> m;
m["hello"]=1;
m["abc"]=2;
cout<<"hello="<<m["hello"]<<endl;
for(auto p=m.begin();p!=m.end();p++) //m.begin为结构体指针
    //指针就->, 结构体就.
cout<<p->first<<":"<<p->second<<endl;
cout<<m.size()//获取长度
```

Fence 11

1.5.8 set

set集合内元素互异，且按从大到小排序

```
//需要有序遍历、找最小最大：用 set
set<int> s; //有序 O(log n)
s.insert(x); //复杂度：O(1)
s.find(x);
s.erase(x); //复杂度：O(1)
//只需要判断存在，或者去重，不需要排序：用 unordered_set
```

```

unordered_set<int> st;
unordered_set<string> words;//哈希表, 无序 O(1)

//初始化
unordered_set<string> words = {"leet", "code"};
unordered_set<string> b(wordDict.begin(), wordDict.end()); //vector<string>&
wordDict

st.count(x) //存在返回1, 不存在返回0; 复杂度: O(1)
if (st.find(x) != st.end()) { //复杂度: O(1)
    cout << "找到了";
}
//遍历
for (auto x : st) {
    cout << x << " ";
}
for (unordered_set<string>::iterator it = st.begin(); it != st.end(); it++) {
    cout << *it << endl;
}

```

Fence 12

1.5.9 其他写法

```

struct Range{
    int l,r;
    bool operator< (const Range &w)const
    {
        return r<w.r;
    }
}range[100]

```

Fence 13

1.6 数据结构模拟

链表

```

int head,e[N],ne[N],idx;
void init(){
    head=-1;
    idx=0;
}
void add_to_head{//将x插入头节点
    e[idx]=x;
    ne[idx]=head;
    head=idx;
    idx++;
}
void add(int k,int x){//将x插入k节点后面
    e[idx]=x;
    ne[idx]=ne[k];
    ne[k]=idx;
    idx++;
}
void remove(int k){//将k后面节点删除
    ne[k]=ne[ne[k]]
}

```

```
}
```

Fence 14

双链表

```
int e[N],l[N],r[N],idx;
void init(){
    r[0]=1,l[1]=0,idx=2;
}
void add(int k,int x){
    e[idx]=x;
    r[idx]=r[k];
    l[idx]=k;
    l[r[k]]=idx;
    r[k]=idx;
} //再左边插入就另k=l[k];
void remove(int k){
    r[l[k]]=r[k];
    l[r[k]]=l[k];
}
```

Fence 15

1.7 kmp

```
int n,m;
char s[M],p[N];
int ne[N];
int main(){
    cin>>n>>p+1>>m>>s+1;
    for(int i=2,j=0;i<=n;i++){
        while(j&&p[i]!=p[j+1]) j=ne[j];
        if(p[i]==p[j+1]) j++;
        ne[i]=j;
    }
    //kmp匹配过程
    for(int i=1,j=0;i<=m;i++){
        while(j&&s[i]!=p[j+1]) j=ne[j];
        if(s[i]==p[j+1]) j++;
        if(j==n){ //匹配成功
            cout<<i-n;
            j=ne[j];
        }
    }
    return 0;
}
```

Fence 16

1.8 并查集

1. 将两个集合合并 2. 询问两个元素是否在一个集合中

```
int n,m;
int p[N];
int find(int x){
```

```

        if(p[x]!=x) p[x]=find(p[x]);
        return p[x];
    }
    int main(){
        cin>>n>>m;
        for(int i=1;i<=n;i++) p[i]=i;
        while(m--){
            char op[2];
            int a,b;
            scanf("%s%d%d",op,&a,&b);
            if(op[0]=='M') p[find(a)]=find(b);
            else{
                if(find(a)==find(b)) puts("YES");
                else puts("No");
            }
        }
        //判断树根 if(p[x]==x)
        //求x集合编号 while(p[x]!=x) x=p[x]
        //合并两个集合 p[x]=y
    }

```

Fence 17

1.9 堆

1. 插入一个数 `heap[++size]=x;up(size)`
2. 求集合中最小值 `heap[1]`
3. 删除最小值 `heap[1]=heap[size];size--;down(1);`
4. 删除任意元素 `heap[k]=heap[size];size--;down(k);up(k);`
5. 修改任意元素 `heap[k]=x;down(k);up(k);`

Fence 18

1.10 大小写转换+字符串操作

```

s[i]=toupper(s[i]); //转大写
s[i]=tolower(s[i]) //转小写

string s,s1,s2;
s2+=s;//拼接字符串
s<s1//按字典序比较
str1.compare(str2)==0;
s.size() s.length()//获取长度
s.substr(n,m);//从第n个位置截m个字符
s.substr(n);//从第n个字符往后的字符全部截取

s.insert(n,s1);//在第n个位置前插入字符串s1
s.erase(1,3);//把1-3字符串删掉

```

```

s.find(s2,m);//从第m个位置搜索字符串s2
s.find("de");//找到返回d第一次出现位置，未找到返回-1
s,rfind("de");//find从左往右，rfind从右往左
//替换
s.replace(1,3,"111111");

```

Fence 19

2. 算法

2.1 素数判定

```

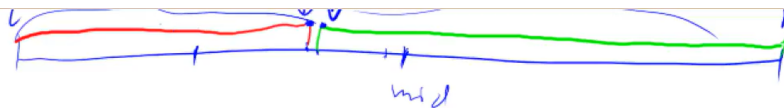
bool isprime(int x){
    if(x==1){
        return 0;
    }
    for(int i=2;i<=sqrt(x);i++){
        if(x%i==0) return 0;
    }
    return 1;
}

```

Fence 20

2.2 二分

整数二分



① $mid = \frac{l+r+1}{2}$
 if (check(mid)) $\begin{cases} \text{true} & [mid, r] : l = mid; \\ \text{false} & [l, mid-1] : r = mid-1 \end{cases}$

② $mid = \frac{l+r}{2}$
 if (check(mid)) $\begin{cases} \text{true} & [l, mid] : r = mid \\ \text{false} & [mid+1, r] : l = mid+1 \end{cases}$

Image 6

```

//区间[l,r]被划分为[l,mid]和[mid+1,r]时使用（绿色边界）
int bsearch1(int l,int r){
    while(l<r){
        int mid=l+r>>1;
        if(check(mid)) r=mid //check()判断是否满足性质
        else l=mid+1;
    }
    return l;
}

```

```

}
//区间[l,r]被划分为[l,mid-1]和[mid,r]时使用（红色边界）
int bsearch2(int l,int r){
    while(l<r){
        int mid=l+r+1>>1;
        if(check(mid)) l=mid //check()判断是否满足性质
        else r=mid-1;
    }
    return l;
}

```

Fence 21

左mid记得+1

浮点数二分

```

int f(double l,double r){
    while(r-l>1e-6){
        double mid=(l+r)/2;
        if(check(mid)) r=mid;
        else l=mid;
    }
    return l
}

```

Fence 22

2.3 高精度

python不用高精

高精度加法

```

#include <bits/stdc++.h>
using namespace std;
vector<int> add(vector<int> &A,vector<int> &B){
    vector<int> C;
    int t=0;
    for(int i=0;i<A.size()||i<B.size();i++){
        if(i<A.size()) t+=A[i];
        if(i<B.size()) t+=B[i];
        C.push_back(t%10);
        t/=10;
    }
    if(t) C.push_back(1);
    return C;
}
int main(){
    string a,b;
    vector<int> A,B;
    cin>>a>>b;
    for(int i=a.size()-1;i>=0;i--) A.push_back(a[i]-'0');
    for(int i=b.size()-1;i>=0;i--) B.push_back(b[i]-'0');
    auto c=add(A,B);
    for(int i=c.size()-1;i>=0;i--) printf("%d",c[i]);
    return 0;
}

```

```
}
```

Fence 23

高精度减法

```
#include <bits/stdc++.h>
using namespace std;
// 判断是否有 A >= B
bool cmp(vector<int> &A,vector<int> &B){
    if (A.size() != B.size()) return A.size() > B.size();
    for (int i = A.size() - 1; i >= 0; i -- )
        if (A[i] != B[i])
            return A[i] > B[i];
    return true;
}
// C = A - B
vector<int> sub(vector<int> &A,vector<int> &B){
    vector<int> C;
    for (int i = 0, t = 0; i < A.size(); i ++ )
    {
        t = A[i] - t;
        if (i < B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        if (t < 0) t = 1;
        else t = 0;
    }

    while (C.size() > 1 && C.back() == 0) C.pop_back(); // 去掉前导0
    return C;
}
int main(){
    string a,b;
    vector<int> A,B;
    cin>>a>>b;
    for(int i=a.size()-1;i>=0;i--) A.push_back(a[i]-'0');
    for(int i=b.size()-1;i>=0;i--) B.push_back(b[i]-'0');
    if(cmp(A,B)){
        auto c=sub(A,B);
        for(int i=c.size()-1;i>=0;i--) printf("%d",c[i]);
    }else{
        auto c=sub(B,A);
        printf("-");
        for(int i=c.size()-1;i>=0;i--) printf("%d",c[i]);
    }
    return 0;
}
```

Fence 24

高精度乘法

大数*小数

```
#include <bits/stdc++.h>
using namespace std;
// C = A * B
```

```

vector<int> mul(vector<int> &A, int b){
    vector<int> C;
    int t = 0;
    for (int i = 0; i < A.size() || t; i ++ )
    {
        if (i < A.size()) t += A[i] * b;
        C.push_back(t % 10);
        t /= 10;
    }
    return C;
}

int main(){
    string a;
    int b;
    cin >> a >> b;
    vector<int> A;
    cin >> a >> b;
    for(int i=a.size()-1; i>=0; i--) A.push_back(a[i]-'0');
    auto c=mul(A,b);
    for(int i=c.size()-1; i>=0; i--) printf("%d", c[i]);
    return 0;
}

```

Fence 25

大数除小数

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
// A / b, 商是C, 余数是r
vector<int> div(vector<int> &A, int b, int &r) // r是引用
{
    vector<int> C; // 商
    r = 0;
    for (int i = A.size() - 1; i >= 0; i -- )
    {
        r = r * 10 + A[i];
        C.push_back(r / b);
        r %= b;
    }
    reverse(C.begin(), C.end());
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

int main()
{
    string a;
    int b;
    cin >> a >> b;
    vector<int> A;
    for (int i = a.size() - 1; i >= 0; i -- ) A.push_back(a[i] - '0');
    int r;
    auto C = div(A, b, r);
}

```

```

    for (int i = c.size() - 1; i >= 0; i -- ) printf("%d", c[i]);
    cout << endl << r << endl;
}

```

Fence 26

2.4 双指针

```

for(int i=0,j=0;i<n;i++){
    while(j<i&&check(i,j)) j++;
    //每道题具体逻辑O(n*n)->O(n)
}
//输出带空格字符串中的每个单词 ad sdf rt
for(int i=0;i<a.length();i++){
    int j=i;
    while(j<n&&j!=" ") j++;
    for(int k=i;k<j;k++) cout<<a[k];
    i=j;
}
//最长连续不重复子序列
for(int i=0,j=0;i<n;i++){
    s[a[i]]++; //s[] 存每个数出现次数
    while(s[a[i]]>1){ //j在右, i在左
        s[a[j]]--;
        j++;
    }
    res=max(res,i-j+1);
}

```

Fence 27

2.5 位运算

```

//1.n的二进制表示中第k位是几
n>>k&1 //n>>k既把第k位移到最后一位 &1是查看最后一位
//二进制表示
int n=10;
for(int k=3;k>=0;k--) cout<<(n>>k&1);

//2.lowbit(x) 返回x最后一个1 1010->10(2) 101000->1000 [返回二进制数]
x & -x //实现lowbit操作
//统计每个数二进制中1的个数
while(x) x-lowbit(x),res++;
//3.原码 0101 反码1010 补码(取反+1) 1011

```

Fence 28

2.6 离散化

a[] 值域很大，但元素个数很少

```

//1.a[]中有重复元素->去重
vector<int> alls; // 存储所有待离散化的值
sort(alls.begin(), alls.end()); // 将所有值排序
alls.erase(unique(alls.begin(), alls.end()), alls.end()); // 去掉重复元素

```

```

//2.离散化
// 二分求出x对应的离散化的值
int find(int x) // 找到第一个大于等于x的位置
{
    int l = 0, r = alls.size() - 1;
    while (l < r)
    {
        int mid = l + r >> 1;
        if (alls[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return r + 1; // 映射到1, 2, ...n
}

//例
int n,m;
int a[N],s[N];
vector<int> alls;
vector<PII> add,query;
int find(int x) // 找到第一个大于等于x的位置
{
    int l = 0, r = alls.size() - 1;
    while (l < r)
    {
        int mid = l + r >> 1;
        if (alls[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return r + 1; // 映射到1, 2, ...n
}

int main()
{
    cin >> n >> m;
    for (int i = 0; i < n; i ++ )
    {
        int x, c; //在x位置上的数+c
        cin >> x >> c;
        add.push_back({x, c});
        alls.push_back(x);
    }
    for (int i = 0; i < m; i ++ )
    {
        int l, r; //询问[l,r]区间和
        cin >> l >> r;
        query.push_back({l, r});
        alls.push_back(l);
        alls.push_back(r);
    }
    // 去重
    sort(alls.begin(), alls.end());
    alls.erase(unique(alls.begin(), alls.end()), alls.end());
    // 处理插入
    for (auto item : add)
    {
        int x = find(item.first);
        a[x] += item.second;
    }
}

```

```

// 预处理前缀和
for (int i = 1; i < a1s.size(); i ++ ) s[i] = s[i - 1] + a[i];
// 处理询问
for (auto item : query){
    int l=find(item.first),r=find(item.second);
    cout<<s[r]-s[l-1]<<endl;
}
return 0;
}

```

Fence 29

2.7 区间合并

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
typedef pair<int, int> PII;
const int N = 100010;

int n;
vector<PII> segs;

void merge(vector<PII> &segs)
{
    vector<PII> res;
    sort(segs.begin(), segs.end());
    int st = -2e9, ed = -2e9;
    for (auto seg : segs)
    {
        if (ed < seg.first)
        {
            if (st != -2e9) res.push_back({st, ed});
            st = seg.first;
            ed = seg.second;
        }
        else ed = max(ed, seg.second);
    }
    if (st != -2e9) res.push_back({st, ed});
    segs = res;
}

int main()
{
    cin >> n;
    for(int i=0;i<n;i++){
        int l,r;
        cin>>l>>r;
        segs.push_back({l,r});
    }
    merge(segs);
    cout<<segs.size()<<endl;
    return 0;
}

```

2.8 Kadane算法

Fence 30

描述：给你一个整数数组 `nums`，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

```
for(int i=1;i<n;i++){
    sum=max(nums[i],sum+nums[i]);
    ans=max(ans,sum);
} //要么在连续的基础上加，要么从头开始

//环形
curMin = min(curMin + x, x);
minSum = min(minSum, curMin); //做一个最大，一个最小
if (maxSum < 0) return maxSum; //特判一下全负情况
return max(maxSum, total - minSum);
```

Fence 31

2.9 前缀和

一维

```
for(int i=1;i<=n;i++){
    cin>>a[i];
}
for(int i=1;i<=n;i++){
    s[i]=s[i-1]+a[i];
}
while(m--){
    cout<<s[r]-s[l-1]; //l到r间数字的和
}
```

Fence 32

二维

```
for(int i =1;i<=n;i++){
    for(int j=1;j<=n;j++){
        s[i][j]=s[i-1][j]+s[i][j-1]-s[i-1][j-1]+a[i][j];
    }
}
while(q--){
    int x1,x2,y1,y2;
    cin>>x1>>x2>>y1>>y2;
    cout<<s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1]; //
}
```

Fence 33

2.10 差分

一维

a是b的前缀和，b是a的差分

```
//让a在[l,r]区间内全部加c->b(l)+c b(r+1)-c
int n,m;
```

```

int a[N],b[N];
void insert(int l,int r,int c){
    b[l]+=c;
    b[r+1]-=c;
}
int main(){
    for(int i=0;i<n;i++) cin>>a[i];
    for(int i=0;i<n;i++) insert(i,i,a[i]); //构建差分;
    while(q--){
        int l,r,c;
        insert(l,r,c);
    }
    for(int i=0;i<n;i++) b[i]+=b[i-1]; //将b构建为前缀和
}

```

Fence 34

二维

```

void insert(int x1,int x2,int y1,int y2,int c){
    b[x1][y1]+=c;
    b[x2-1][y1]-=c;
    b[x1][y2-1]-=c;
    b[x2+1][y2+1]+=c;
}
int main(){
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            cin>>a[i][j];
        }
    }
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            insert(i,j,i,j,a[i][j]);
        }
    }
    while(q--){
        insert(x1,x2,y1,y2,c);
    }
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            b[i][j]+=b[i-1][j]+b[i][j-1]-b[i-1][j-1];
        }
    }
}

```

Fence 35

3. 图论

3.1 dijkstra

朴素dijkstra

```

#include <bits/stdc++.h>
using namespace std;
const int N=10000,M=100;

```

```

int n,m;
int g[N][N];
int dist[N];
bool st[N];
int dijkstra(){
    memset(dist,0x3f,sizeof dist);
    dist[1]=0;
    for(int i=0;i<n;i++){
        int t=-1;
        for(int j=1;j<=n;j++){
            if(!st[j]&&(t==-1||dist[t]>dist[j])){
                t=j;
            }
        }
        st[t]=true;
        for(int j=1;j<=n;j++){
            dist[j]=min(dist[j],dist[t]+g[t][j]);
        }
    }
    if(dist[n]==0x3f3f3f3f) return -1;
    return dist[n];
}
int main(){
    cin>>n>>m;
    memset(g,0x3f,sizeof g);
    while(m--){
        int a,b,c;
        cin>>a>>b>>c;
        g[a][b]=min(g[a][b],c);
    }
    int t=dijkstra();
    cout<<t;
    return 0;
}

```

Fence 36

堆优化

```

#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> PII;
const int N=10000,M=100;
int n,m;
int h[N],w[N],e[N],ne[N],idx;//w为权重
int dist[N];
bool st[N];
void add(int a,int b,int c){
    e[idx]=b;w[idx]=c;ne[idx]=h[a];h[a]=idx++;
}
int dijkstra(){
    memset(dist,0x3f,sizeof dist);
    dist[1]=0;
    priority_queue<PII,vector<PII>,greater<PII>> heap;
    heap.push({0,1});
}

```

```

while(heap.size()){
    auto t=heap.top();
    heap.pop();
    int ver=t.second,distance=t.first;
    if(st[ver]) continue;
    st[ver]=true;
    for(int i=h[ver];i!=-1;i=ne[i]){
        int j=e[i];
        if(dist[j]>distance+w[i]){
            dist[j]=distance+w[i];
            heap.push({dist[j],j});
        }
    }
}

if(dist[n]==0x3f3f3f3f) return -1;
return dist[n];
}

int main(){
    cin>>n>>m;
    memset(h,-1,sizeof h);
    while(m--){
        int a,b,c;
        cin>>a>>b>>c;
        add(a,b,c);
    }
    int t=dijkstra();
    cout<<t;
    return 0;
}

```

Fence 37

3.2 bellman_ford

```

#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> PII;
const int N=10000,M=100;
int n,m,k;
int dist[N],backup[N];
struct Edge{
    int a,b,w;
}edge[M];

int bellman_ford(){
    memset(dist,0x3f,sizeof dist);
    dist[1]=0;
    for(int i=0;i<k;i++){
        memcpy(backup,dist,sizeof dist);
        for(int j=0;j<m;j++){
            int a=edge[j].a,b=edge[j].b,w=edge[j].w;
            dist[b]=min(dist[b],backup[a]+w);
        }
    }
}

```

```

        if(dist[n]>0x3f3f3f3f/2) return -1;
        return dist[n];
    }
    int main(){
        cin>>n>>m>>k;
        for(int i=0;i<m;i++){
            int a,b,w;
            cin>>a>>b>>w;
            edge[i]={a,b,w};
        }
        int t=bellman_ford();
        if(t==-1) puts("impossible");
        else cout<<t;
        return 0;
    }

```

Fence 38

3.3 spfa 一般可以用spfa替代di j，但是被卡就换

```

#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> PII;
const int N=10000,M=100;
int n,m;
int h[N],w[N],e[N],ne[N],idx;//w为权重
int dist[N];
bool st[N];
void add(int a,int b,int c){
    e[idx]=b;w[idx]=c;ne[idx]=h[a];h[a]=idx++;
}
int spfa(){
    memset(dist,0x3f,sizeof dist);
    dist[1]=0;

    queue<int> q;
    q.push(1);
    st[1]=true;

    while(q.size()){
        int t=q.front();
        q.pop();
        st[t]=false;
        for(int i=h[t];i!=-1;i=ne[i]){
            int j=e[i];
            if(dist[j]>dist[t]+w[i]){
                dist[j]=dist[t]+w[i];
                if(!st[j]){
                    q.push(j);
                    st[j]=true;
                }
            }
        }
    }
}

```

```

        if(dist[n]==0x3f3f3f) return -1;
        return dist[n];
    }
    int main(){
        cin>>n>>m;
        memset(h,-1,sizeof h);
        while(m--){
            int a,b,c;
            cin>>a>>b>>c;
            add(a,b,c);
        }
        int t=spfa();
        if(t==-1) puts("impossible");
        else cout<<t;
        return 0;
    }

```

Fence 39

spfa判断负环

```

#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> PII;
const int N=10000;
int n,m;
int h[N],w[N],e[N],ne[N],idx;//w为权重
int dist[N],cnt[N];
bool st[N];
void add(int a,int b,int c){
    e[idx]=b;w[idx]=c;ne[idx]=h[a];h[a]=idx++;
}
bool spfa(){
    queue<int> q;
    for(int i=1;i<=n;i++){
        q.push(i);    //把所有点放入队列
        st[i]=true;
    }

    while(q.size()){
        int t=q.front();
        q.pop();
        st[t]=false;
        for(int i=h[t];i!=-1;i=ne[i]){
            int j=e[i];
            if(dist[j]>dist[t]+w[i]){
                dist[j]=dist[t]+w[i];
                cnt[j]=cnt[t]+1;
                if(cnt[j]>=n) return true;
                if(!st[j]){
                    q.push(j);
                    st[j]=true;
                }
            }
        }
    }
    return false;
}

```

```

    }
    }
}

return false;

}
int main(){
    cin>>n>>m;
    memset(h,-1,sizeof h);
    while(m--){
        int a,b,c;
        cin>>a>>b>>c;
        add(a,b,c);
    }
    if(spfa()) puts("yes");
    else puts("no");
    return 0;
}

```

Fence 40

3.4 floyd

```

#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> PII;
const int N=10000,INF=1e9;
int n,m,q;
int d[N][N];
void floyd(){
    for(int k=1;k<=n;k++){
        for(int i=1;i<=n;i++){
            for(int j=1;j<=n;j++){
                d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
            }
        }
    }
}
int main(){
    cin>>n>>m>>q;
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            if(i==j) d[i][j]=0;
            else d[i][j]=INF;
        }
    }
    while(m--){
        int a,b,w;
        cin>>a>>b>>w;
        d[a][b]=min(d[a][b],w);
    }
    floyd();
    while(q--){
        int a,b;

```

```

        cin>>a>>b;
        if(d[a][b]>INF/2) puts("impossible");
        else cout<<d[a][b];
    }
    return 0;
}

```

Fence 41

4. 搜索

4.1 dfs

```

#include <bits/stdc++.h>
using namespace std;
const int N=1010;
int n;
int path[N];
bool st[N];
void dfs(int u){
    if(u==0){
        for(int i=0;i<n;i++) cout<<path[i];
        return;
    }
    for(int i=1;i<=n;i++){
        if(!st[i]){
            path[u]=i;
            st[u]=true;
            dfs(u);
            st[u]=false;
        }
    }
}
int main(){
    cin>>n;
    dfs(0);
    return 0;
}

```

Fence 42

```

11 dfs(11 x){    //记忆化搜索
    if(x<2){
        return 0;
    }
    if(mp.count(x)){
        return mp[x];
    }
    mp[x]=x+dfs(x/2)+dfs((x+1)/2)
    return mp[x];
}

```

Fence 43

4.2 bfs

最短路

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> PII;
const int N=10,M=N*N;
#define x first
#define y second
int n,m;
int g[N][N];
PII q[M];
PII pre[N][N];
void bfs(int sx,int sy){
    int dx[4]={-1,0,1,0},dy[4]={0,1,0,-1};
    int hh=0,tt=0;
    q[0]={sx,sy};

    memset(pre,-1,sizeof pre);
    pre[sx][sy]={0,0};
    while(hh<=tt){
        PII t=q[hh++];
        for(int i=0;i<4;i++){
            int a=t.x+dx[i],b=t.y+dy[i];

            if(a<0||a>=n||b<0||b>=n) continue;
            if(g[a][b]) continue; //是墙
            if(pre[a][b].x!=-1) continue;

            q[++tt]={a,b};
            pre[a][b]=t;
        }
    }
}

int main(){
    cin>>n;
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            cin>>g[i][j];
        }
    }
    // 逆向搜索策略：从终点搜回起点(fang)
    bfs(n-1,n-1);
    // 路径还原：从起点(0,0)顺着 pre 指针找回终点
    PII end(0,0);
    while(true){
        cout<<end.x<<" "<<end.y<<endl;
        if(end.x==n-1&&end.y==n-1) break;
        end=pre[end.x][end.y];
    }
    return 0;
}
```

```
#include <bits/stdc++.h>
using namespace std;
unordered_map<string,int> dist;
unordered_map<string,pair<char,string>> pre;
queue<string> q;
char g[2][4];
void set1(string start){
    for(int i=0;i<4;i++) g[0][i]=start[i];
    for(int i=3,j=4;i>=0;i--,j++) g[1][i]=start[j];
}
string get(){
    string res;
    for(int i=0;i<4;i++) res+=g[0][i];
    for(int i=3;i>=0;i--) res+=g[1][i];
    return res;
}
string move1(string start){
    set1(start);
    for(int i=0;i<4;i++) swap(g[0][i],g[1][i]);
    return get();
}
string move2(string start){
    set1(start);
    char v0=g[0][3],v1=g[1][3];
    for(int i=3;i>0;i--){
        g[1][i]=g[1][i-1];
        g[0][i]=g[0][i-1];
    }
    g[0][0]=v0,g[1][0]=v1;
    return get();
}
string move3(string start){
    set1(start);
    char t=g[0][2];
    g[0][2]=g[0][1];
    g[0][1]=g[1][1];
    g[1][1]=g[1][2];
    g[1][2]=t;
    return get();
}
void bfs(string start,string end){
    if(start==end) return;
    q.push(start);
    dist[start]=0;
    while(q.size()){
        auto t=q.front();
        q.pop();
        string m[3];
        m[0]=move1(t);
        m[1]=move2(t);
        m[2]=move3(t);
        for(int i=0;i<3;i++){
            string m1=m[i];
```

```

        if(dist.count(m1)==0){
            dist[m1]=dist[t]+1;
            pre[m1]={char(i+'A'),t};
            if(m1==end) break;
            q.push(m1);
        }
    }
}

int main(){
    string start,end;
    int x;
    for(int i=0;i<8;i++){
        cin>>x;
        end+=char(x+'0');
    }
    for(int i=0;i<8;i++){
        start+=char(i+'1');
    }
    bfs(start,end);
    cout<<dist[end]<<endl;
    string res;
    while(end!=start){
        res+=pre[end].first;
        end=pre[end].second;
    }
    reverse(res.begin(),res.end());
    if(res.size()) cout<<res<<endl;
    return 0;
}

```

Fence 45

双向广搜

双向广搜是最小步数模型的优化策略

```

#include <bits/stdc++.h>
using namespace std;
const int N=6;
int n;
string a[N],b[N];
int extend(queue<string> &q,unordered_map<string,int>
&da,unordered_map<string,int> &db,string a[],string b[]){
    string t=q.front();
    q.pop();
    for(int i=0;i<t.size();i++){
        for(int j=0;j<n;j++){
            if(t.substr(i,a[j].size())==a[j]){
                string state=t.substr(0,i)+b[j]+t.substr(i+a[j].size());
                if(da.count(state)) continue;//走过这个状态, 跳过
                //map中的count: 查找有无对应的key, 有返1无返0
                if(db.count(state)) return da[t]+1+db[state];
                //两端碰上了, 记录
                da[state]=da[t]+1;
            }
        }
    }
    q.push(t);
    return -1;
}

```

```

        q.push(state);
    }
}
return 11;
}
int bfs(string A,string B){
    queue<string> qa,qb;//存状态
    unordered_map<string,int> da,db;
    //记录每个状态离起点or终点距离
    qa.push(A),da[A]=0;
    qb.push(B),db[B]=0;
    while(qa.size() && qb.size()){
        int t;
        if(qa.size() <= qb.size()) t=extend(qa,da,db,a,b);
        else t=extend(qb,db,da,b,a);
        if(t <= 10) return t;
    }
    return 11;
}
int main(){
    string A,B;
    cin>>A>>B;
    while(cin>>a[n]>>b[n]) n++;
    int step=bfs(A,B);
    if(step>10) cout<<"no impossible"<<endl;
    else cout<<step<<endl;
    return 0;
}

```

Fence 46

5. 动态规划

5.1 背包dp

01背包

```

int f[N];
int n,m;
int v[N],w[N];
int main(){
    for(int i=1;i<=n;i++){
        cin>>v[i]>>w[i];
    }
    for(int i=1;i<=n;i++){
        for(int j=m;j>=v[i];j--){
            f[j]=max(f[j],f[j-v[i]]+w[i]);
        }
    }
    cout<<f[m]<<endl;
    return 0;
}

```

Fence 47

```

int n,v,M;
int f[N][N];
int main(){
    cin>>n>>v>>M;
    for(int i=1;i<=n;i++){
        int v,m,w;
        cin>>v>>m>>w;
        for(int j=v;j>=v;j--){
            for(int k=M;k>=m;k--){
                f[j][k]=max(f[j][k],f[j-v][k-m]+w);
            }
        }
    }
    cout<<f[v][M];
}

```

Fence 48

完全背包

```

int v[N],w[N];
int f[N];
int n,m;
int main(){
    for(int i=1;i<=n;i++) cin>>v[i]>>w[i];
    for(int i=1;i<=n;i++){
        for(int j=v[i];j<=m;j++){
            f[j]=max(f[j],f[j-v[i]]+w[i]);
        }
    }
    cout<<f[m]<<endl;
}

```

//没给w

//题目描述: 给你一个整数数组 **coins** , 表示不同面额的硬币; 以及一个整数 **amount** , 表示总金额。计算并返回可以凑成总金额所需的 最少的硬币个数 。如果没有任何一种硬币组合能组成总金额, 返回 **-1** 。硬币数量无限

```

int coinChange(vector<int>& coins, int amount) {
    int n=coins.size();
    const int INF = 1e9;
    vector<int> f(amount+1,INF);
    f[0]=0;
    for(int i=0;i<n;i++){
        for(int j=coins[i];j<=amount;j++){
            f[j]=min(f[j],f[j-coins[i]]+1);
        }
    }
    if(f[amount]==INF){
        return -1;
    }
    return f[amount];
}

```

原来的物品数: n

拆分后的物品数: $N=n*\log(\text{每个物品的数量 } s)$

```
int f[N];
int n,m;
int v[N],w[N];
int main(){
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        int a,b,s;
        cin>>a>>b>>s;
        int k=1;
        while(k<=s){
            cnt++;
            v[cnt]=a*k;
            w[cnt]=b*k;
            s-=k;
            k*=2;
        }
        if(s>0){
            cnt++;
            v[cnt]=a*s;
            w[cnt]=b*s;
        }
    }
    n=cnt;
    for(int i=1;i<=n;i++){
        for(int j=m;j>=v[i];j--){
            f[j]=max(f[j],f[j-v[i]]+w[i]);
        }
    }
    cout<<f[m]<<endl;
}
```

```
int n,m;
int v[N][N],w[N][N],s[N];
int f[N];
int main(){
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>s[i];
        for(int j=0;j<s[i];j++){
            cin>>v[i][j]>>w[i][j];
        }
    }
    for(int i=1;i<=n;i++){
        for(int j=m;j>=0;j--){
            for(int k=0;k<s[i];k++){
                if(j>=v[i][k]){
                    f[j]=max(f[j],f[j-v[i][k]]+w[i][k])
                }
            }
        }
    }
}
```

```

    }
    }
}
cour<<f[m]<<endl;
return 0;
}

```

Fence 51

5.2 数字三角形模型

给定一个如下图所示的数字三角形，从顶部出发，在每一结点可以选择移动至其左下方的结点或移动至其右下方的结点，一直走到底层，要求找出一条路径，使路径上的数字的和最大。

```

      7
     3 8
    8 1 0
   2 7 4 4
  4 5 2 6 5

```

Image 7

```

class Solution {
public:
    int minimumTotal(vector<vector<int>>& triangle) {
        int n = triangle.size();
        vector<int> f(n, 0); //一维优化，因为只依赖下一层
        for (int j = 0; j < n; j++) {
            f[j] = triangle[n - 1][j];
        }
        for (int i = n - 2; i >= 0; i--) {
            for (int j = 0; j <= i; j++) {
                f[j] = min(f[j], f[j + 1]) + triangle[i][j];
            }
        }
        return f[0];
    }
};

```

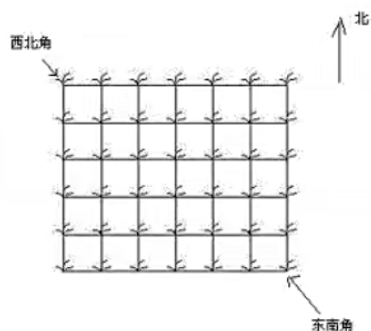
Fence 52

她来到一片有网格状道路的矩形花生地(如下图)，从西北角进去，东南角出来。

地里每个道路的交叉点上都有种着一株花生苗，上面有若干颗花生，经过一株花生苗就能摘走该它上面所有的花生。

Hello Kitty只能向东或向南走，不能向西或向北走。

问Hello Kitty最多能够摘到多少颗花生。



$f[i, j]$: 从 (1, 1) 走到 (i, j) 的所有路线最大值

```
int n;
int w[N][N];
int f[N][N];
int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            scanf("%d", &w[i][j]);
    //求最大值不用考虑初始化
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            f[i][j]=max(f[i-1][j],f[i][j-1])+w[i][j];
        }
    }

    //求最小值要考虑初始化
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (i == 1 && j == 1) f[i][j] = w[i][j];    // 特判左上角
            else
            {
                //特殊处理第一行第一列，因为之外为0
                f[i][j] = INF;
                if (i > 1) f[i][j] = min(f[i][j], f[i - 1][j] + w[i][j]);    //
                //只有不在第一行的时候，才可以从上面过来
                if (j > 1) f[i][j] = min(f[i][j], f[i][j - 1] + w[i][j]);    //
                //只有不在第一列的时候，才可以从左边过来
            }
    printf("%d\n", f[n][n]);
    return 0;
}
```

Fence 53

一维优化版

```
//求最大值
f[1] = 0;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        f[j] = max(f[j], f[j - 1]) + w[i][j];
    }
}
cout << f[m] << endl;
//求最小值
for (int i = 0; i < n; i++) {    //f[j]代表从上面来，f[j-1]代表从左边来
    for (int j = 0; j < m; j++) {
        if (i == 0 && j == 0) {
            f[j] = grid[i][j];
        } else if (i == 0) {
            f[j] = f[j - 1] + grid[i][j];
        }
    }
}
```

```

        } else if (j == 0) {
            f[j] = f[j] + grid[i][j];
        } else {
            f[j] = min(f[j], f[j - 1]) + grid[i][j];
        }
    }
}
return f[m - 1];

```

Fence 54

5.3 字符串dp

描述：给你两个单词 `word1` 和 `word2`，请返回将 `word1` 转换成 `word2` 所使用的最少操作数。

可以对一个单词进行如下三种操作：1. 插入一个字符 2. 删除一个字符 3. 替换一个字符

`f[i][j]` -> 把 `word1` 的前 `i` 个字符，变成 `word2` 的前 `j` 个字符，最少需要多少次操作。

```

class Solution {
public:
    int minDistance(string word1, string word2) {
        int n = word1.size();
        int m = word2.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
        for (int i = 0; i <= n; i++) {
            dp[i][0] = i; //word2 是空串,就只能删
        }
        for (int j = 0; j <= m; j++) {
            dp[0][j] = j; //word1 是空串,就只能添
        }
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (word1[i - 1] == word2[j - 1]) { //最后一个字符相等
                    dp[i][j] = dp[i - 1][j - 1];
                } else { //最后一个字符不相等
                    dp[i][j] = min({
                        dp[i - 1][j] + 1, // 删除
                        dp[i][j - 1] + 1, // 插入
                        dp[i - 1][j - 1] + 1 // 替换
                    });
                }
            }
        }
        return dp[n][m];
    }
};

```

Fence 55

最长上升子序列模型

001号分支

```

//单向
for(int i=1;i<=n;i++){
    f[i]=1;
    for(int j=1;j<i;j++){ //如果vector记得改成从0开始
        if(a[j]<a[i]){
            f[i]=max(f[j]+1,f[i]);
        }
    }
}

int res=0;
for(int i=1;i<=n;i++){
    res=max(res,f[i]);
}

//大数据时优化算法
const int N=10010;
int n;
int a[N];
int q[N];
int main(){
    cin>>n;
    int res=0;
    for(int i=0;i<n;i++) cin>>a[i];
    int len=0;
    for(int i=0;i<n;i++){
        q[0]=-1e9;
        int l=0,r=len;
        while(l<r){
            int mid=l+r+1>>1;
            if(q[mid]<a[i]) l=mid;
            else r=mid-1;
        }
        q[r+1]=a[i];
        len=max(len,r+1);
    }
    cout<<len<<endl;
}

//双向
for(int i=1;i<=n;i++){
    f[i]=1;
    for(int j=1;j<i;j++){
        if(a[j]<a[i]){
            f[i]=max(f[j]+1,f[i]);
        }
    }
}

for(int i=n;i>=1;i--){
    g[i]=1;
    for(int j=n;j>i;j--){
        if(a[j]<a[i]){
            g[i]=max(g[j]+1,g[i]);
        }
    }
}

int res=0;
for(int i=1;i<=n;i++){

```

```

        res=max(res,f[i]+g[i]-1);
    }

```

Fence 56

002号分支（以一边排序，另一边求最大上升子序列）

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int,int> PII;
ll n;
const int N=200005;
ll f[N];
PII q[N];
int main(){
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>q[i].first>>q[i].second;
    }
    sort(q+1,q+n+1); //务必注意，下标从1开始要加1;
    ll res=0;
    for(int i=1;i<=n;i++){
        f[i]=1;
        for(int j=1;j<i;j++){
            if(q[i].second>q[j].second){
                f[i]=max(f[i],f[j]+1);
            }
        }
        res=max(res,f[i]);
    }
    cout<<res<<endl;
    return 0;
}

```

Fence 57

003分支（最大上升子序列和）

```

for(int i=1;i<=n;i++){
    f[i]=a[i];
    for(int j=1;j<i;j++){
        if(a[i]>a[j]){
            f[i]=max(f[j]+a[i],f[i]);
        }
    }
}
}

```

Fence 58

004分支（最长上升子序列2-结合贪心）

5.4 区间dp

001矩阵连乘

```

#include <bits/stdc++.h>
using namespace std;

```

```

const long long INF = 1e18;
int main() {
    int n;
    cin >> n;
    vector<long long> p(n + 1);
    for (int i = 0; i <= n; i++) {
        cin >> p[i];
    }
    vector<vector<long long>> dp(n + 1, vector<long long>(n + 1, 0));

    // len 表示当前区间长度，也就是矩阵个数
    for (int len = 2; len <= n; len++) {
        for (int i = 1; i + len - 1 <= n; i++) {
            int j = i + len - 1;
            dp[i][j] = INF;

            // 枚举最后一次断开的位置 k
            for (int k = i; k < j; k++) {
                long long cost = dp[i][k] + dp[k + 1][j] + p[i - 1] * p[k] *
p[j];
                dp[i][j] = min(dp[i][j], cost);
            }
        }
    }
    cout << dp[1][n] << '\n';
    return 0;
}

```

Fence 59

002 环形区间

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N=410,INF=0x3f3f3f;
int n;
int s[N], w[N];
int f[N][N], g[N][N];
//环形区间合并+求最大值+最小值
int main()
{
    cin >> n;
    for (int i = 1; i <= n; i++)
    {
        cin >> w[i];
        w[i + n] = w[i];
    }
    for (int i = 1; i <= n * 2; i++) s[i] = s[i - 1] + w[i];
    memset(f, 0x3f, sizeof f);
    memset(g, -0x3f, sizeof g);
    for (int len = 1; len <= n; len++)
        for (int l = 1; l + len - 1 <= n * 2; l++)
        {
            int r = l + len - 1;
            if (len == 1) f[l][r] = g[l][r] = 0;

```

```

else
{
    for (int k = 1; k < r; k++)
    {
        f[l][r] = min(f[l][r], f[l][k] + f[k + 1][r] + s[r] - s[l] -
1]);
        g[l][r] = max(g[l][r], g[l][k] + g[k + 1][r] + s[r] - s[l] -
1]);
    }
}

int minv = INF, maxv = -INF;
for (int i = 1; i <= n; i++)
{
    minv = min(minv, f[i][i + n - 1]);
    maxv = max(maxv, g[i][i + n - 1]);
}
cout << minv << endl << maxv << endl;
return 0;
}

```

Fence 60

6. 贪心

求一个点到其他所有点距离最小：曼哈顿距离两维度互不干扰，可以分别贪心；欧几里得距离-》费马点

6.1 区间选点(最大不相交区间个数)

描述：用最少的点涵盖所有给出的区间

活动安排问题

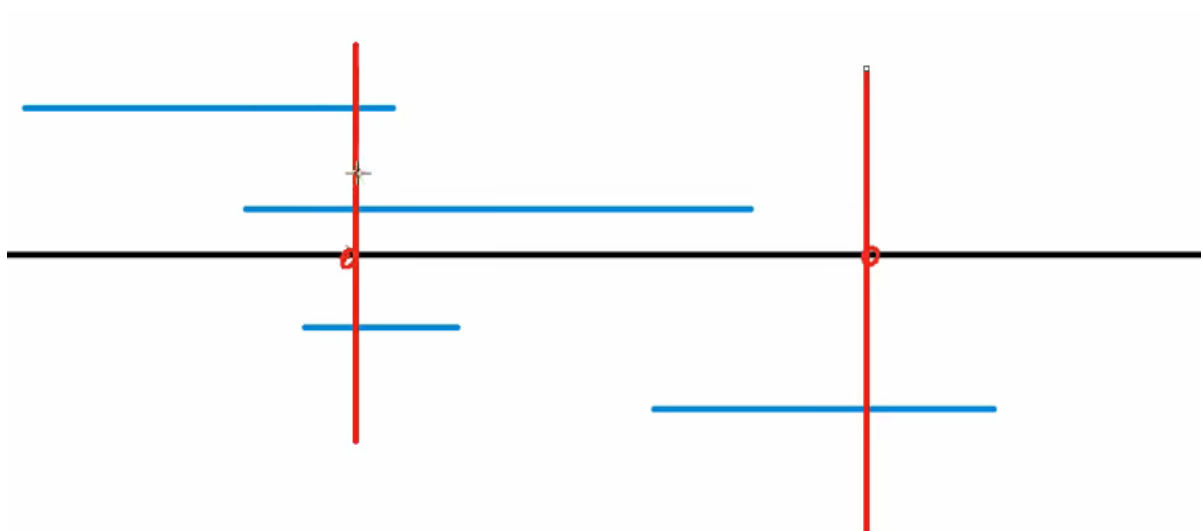


Image 9

```

#include <bits/stdc++.h>
using namespace std;
const int N=1010;
int n;
struct Range{
    int l,r;
}

```

```

bool operator< (const Range &w) const
{
    return r<w.r;
} // 结构体自定义比较
} range[N];
int main(){
    cin>>n;
    for(int i=0;i<n;i++){
        int l,r;
        cin>>l>>r;
        range[i]={l,r};
    }
    sort(range,range+n);
    int res=0,ed=-2e9; // 点数、右端点
    for(int i=0;i<n;i++){
        if(range[i].l>ed){
            res++; // 超过右端点，新开分组
            ed=range[i].r;
        }
    }
    cout<<res;
    return 0;
}

```

Fence 61

6.2 区间分组

描述：分成若干组，使得组内区间两两无交集，且数组尽可能小



Image 10

```

#include <bits/stdc++.h>
using namespace std;
const int N=1010;
int n;
struct Range{
    int l,r;
    bool operator< (const Range &w) const
    {
        return l<w.l;
    } // 结构体自定义比较
} range[N];
int main(){
    cin>>n;
    for(int i=0;i<n;i++){
        int l,r;
        cin>>l>>r;
    }
}

```

```

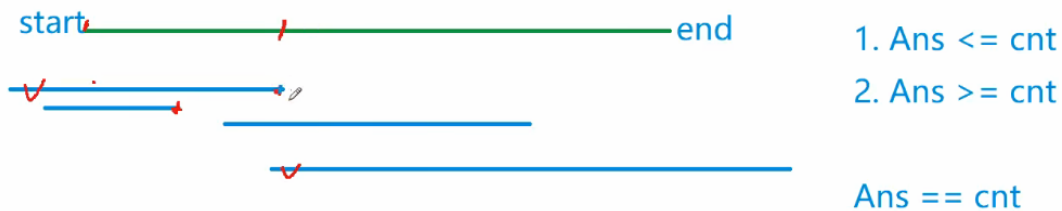
        range[i]={l,r};
    }
    sort(range,range+n);
    priority_queue<int,vector<int>,greater<int> > heap; //小根堆确保找到最小右端点
    for(int i=0;i<n;i++){
        auto r=range[i]; //c++11 有auto关键字
        if(heap.empty()||heap.top()>=r.l) heap.push(r.r); //最小的都有重叠，需要开
新组
        else{
            int t=heap.top(); //当前区间可以放到某一组里
            heap.pop();
            heap.push(r.r);
        }
    }
    cout<<heap.size();
    return 0;
}

```

Fence 62

6.3 区间覆盖

描述：在指定的N个区间选尽量少的区间[a, b]将一个区间[s, t]覆盖。输出区间数，不能就输出-1



1. 将所有区间按左端点从小到大排序
2. 从前往后依次枚举每个区间，在所有能覆盖start的区间中，选择右端点最大的区间然后将start更新成右端点的最大值

www.ac

Image 11

```

#include <bits/stdc++.h>
using namespace std;
const int N=100010;
struct Range{
    int l,r;
    bool operator< (const Range &w)const{
        return l<w.l;
    }
};
int n;
int Range[N];
int main(){
    int st, ed;
    scanf("%d%d", &st, &ed);

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
    {
        int l, r;

```

```

scanf("%d%d", &l, &r);
range[i] = {l, r};
}
sort(range, range + n);
int res = 0;
for (int i = 0; i < n; i++)
{
    int j = i, r = -2e9; //双指针算法扫描一遍
    while (j < n && range[j].l <= st)
    {
        r = max(r, range[j].r); //在左端点达标情况下，右端点尽可能长
        j++;
    }
    if (r < st)
    {
        res = -1;
        break;
    }
    res++;
    if (r >= ed) break;
    st = r;
    i = j - 1;
}
printf("%d\n", res);
return 0;
}

```

Fence 63

6.4 果子合并（哈夫曼树）

描述：每次合并俩堆果子，问合并所有果子所需最小体力值

```

int n;
scanf("%d",&n);
priority_queue<int ,vector<int>,greater<int>> heap; //定义小根堆
while(n--){
    int x;
    scanf("%d",&x);
    heap.push(x);
}
int res=0;
while(heap.size()>1){
    int a=heap.top();heap.pop(); //取出最小的两个值
    int b=heap.top();heap.pop();
    res+=a+b;
    heap.push(a+b)
}
printf("%d\n",res);
return 0

```

Fence 64

6.5 跳跃游戏

```
bool canJump(vector<int>& nums) {
    int res=0;
    for(int i=0;i<nums.size()-1;i++){
        if (i > res) return false; //注意有些地方跳不到
        res=max(i+nums[i],res);
    }
    if(res>=nums.size()-1){
        return true;
    }else{
        return false;
    }
}
```

Fence 65

排序不等式

1. 从小到大排序 `sort(a, a+n);`
2. (绝对值) 据每个点距离最小, 奇数: 中位数 偶数: 中间两数中间

```
sort(a,a+n);
for(int i=0;i<n;i++) res+=abs(a[i]-a[n/2]);
cout<<res;
```

Fence 66

推函数式

国王游戏/奶牛杂技

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> PII;
const int N=50050;
PII cow[N];
int w[N],s[N];
int n;
int main(){
    cin>>n;
    for(int i=0;i<n;i++){
        cin>>w[i]>>s[i];
        cow[i]={w[i]+s[i],w[i]};
    }
    sort(cow,cow+n); //贪心关键->按w+s排序
    int res=-2e9,sum=0;
    for(int i=0;i<n;i++){
        int w=cow[i].second,s=cow[i].first-w;
        res=max(res,sum-s);
        sum+=w;
    }
    cout<<res;
    return 0;
}
```

Fence 67

